

Lab 2

Multi-threaded Programming with Blocking I/O

2.1 Objectives

This lab is to learn about and gain practical experience in multi-threaded programming in a general Linux environment. A single-thread implementation using blocking I/O to request a resource across the network is provided. Students are asked to reduce the latency of this operation by sending out multiple requests simultaneously to different machines by using the `pthread` library. After this lab, students will

- Design and implement a multi-threaded program using the `pthread` library
- Use multi-threading to reduce the latency of a program.
- Utilize blocking I/O APIs of `cURL` library to request resources across a network.

2.2 Starter Files

The starter files are on GitHub at url: https://git.uwaterloo.ca/ece252-1269/starter_codes/-/tree/main/lab2. It contains the following sub-directories where we have example code to help you get started:

- the `cURL` demonstrates how to use `cURL` to fetch an image segment from the lab web server;
- the `fn_ptr` demonstrates how C function pointers work;
- the `getopt` demonstrates how to parse command line options;
- the `pthread` demonstrates how to create two threads where each thread takes multiple input parameters and return multiple values; and

- the [times](#) provides helper functions to profile program execution times.

Using the code in the starter files is permitted and will not be considered as plagiarism.

2.3 Lab project

2.3.1 Problem Statement

In the previous lab, a set of horizontal strips of a whole PNG image file were stored on a disk and you were asked to restore the whole image from these strips. In this lab, the horizontal image segments are on the web. I have three 400×300 pictures (whole images) on three web servers. When you ask a web server to send you a picture, the web server crops the picture into fifty 400×6 equally sized horizontal strips¹. The web server assigns a sequence number to each strip from top to bottom starting from 0 and increments by 1². Then the web server sleeps for a random time and then sends out a random strip in a simple PNG format that we assumed in lab1. That is the horizontal strip PNG image segment consists one IHDR chunk, one IDAT chunk and one IEND chunk (see Figure 1.2a). The PNG segment is an 8-bit RGBA/color image (see Table 1.1 for details). The web server uses an HTTP response header that includes the sequence number to tell you which strip it sends to you. The HTTP response header has the format of "X-Ece252-Fragment: M " where $M \in [0, 49]$. To request a random horizontal strip of picture N , where $N \in [1, 3]$, use the following URL: `http://machine:2520/image?img=N`, where

machine is one of the following:

- `ece252-1.uwaterloo.ca`,
- `ece252-2.uwaterloo.ca`, and
- `ece252-3.uwaterloo.ca`.

For example, when you request data from the following URL:

`http://ece252-1.uwaterloo.ca:2520/image?img=1`,

you may receive a random horizontal strip of picture 1. Assume this random strip you receive is the third horizontal strip (from top to bottom of the original picture), the received HTTP header will contain "X-Ece252-Fragment: 2". The received data will be the image segment in PNG format. You may use the browser to view a random horizontal strip of the PNG image the server sends. You will notice the same URL displays a different image strip every time you hit enter to refresh the page. Each strip has the same dimensions of 400×6 pixels and is in PNG format.

¹Each image segment will have a size less than 8KB.

²The first horizontal strip has a sequence number of 0, the second strip has a sequence number of 1. The sequence number increments by 1 from top to bottom and the last strip has a sequence number of 49.

Your objective is to request all horizontal strips of a picture from the server and then concatenate these strips to restore the original picture. Because every time the server sends a random strip, if you use a loop to keep requesting a random strip from a server, you may receive the same strip multiple times before you receive all the fifty distinct strips. Due to the randomness, it will take a variable amount of time to get all the strips you need to restore the original picture.

2.3.2 Requirements

Use the `pthread` library, design and implement a threaded program to request all image segments from a web server by using blocking I/O and concatenate these segments together to form the whole image.

The provided starter code `main_write_header_cb.c` under `cURL` directory is a single-threaded implementation which uses `libcurl` blocking I/O function `curl_easy_perform()` to fetch one random horizontal strip of picture 1 from one of the web servers into memory and then output the received image segment to a PNG file. Your program should repeatedly fetch the image strips until you have them all. Recall because every time you get a random strip, the amount of time to get all the fifty distinct strips of a picture varies.

A very inefficient approach is to use a single-threaded loop to keep fetching until you get all fifty distinct strips of a picture and paste them together. You will notice the blocking I/O operation is the main cause of the latency. Your program will be blocked while each time waiting for the `curl_easy_perform()` to finish. One way to reduce the latency of this operation is to send out multiple blocking I/O requests simultaneously (to different machines) by using `pthread`. You will use this approach to reduce the latency in this lab ³.

Your program should create as many threads as specified by the user command line input, and distribute the work among the three provided servers. Make sure all of your library (standard `glibc` and `libcurl`) calls are *thread-safe* (for `glibc`, e.g. `man 3 printf` to look at the documentation). Name your executable as `paster`. The behaviour of the command `paster` is given in the following section.

The provided three pictures on the server are for you to test your program. Your program should work for all these pictures. You may want to reuse part of your lab1 code to paste the received image segments together.

2.3.3 Man page of `paster`

NAME

paster - pasting downloaded png files together by using multiple threads and

³Asynchronous I/O is another method to reduce the latency and we will explore it in lab5.

blocking I/O through libcurl.

SYNOPSIS

paster [OPTION]...

DESCRIPTION

With no options, the command retrieves all horizontal image segments of picture 1 from <http://ece252-1.uwaterloo.ca:2520/image?img=1> and paste all distinct segments received from top to bottom in the order of the image segment sequence number. Output the pasted image to disk and name it all.png.

-t=NUM

create NUM threads simultaneously requesting random image segments from multiple lab web servers. When this option is not specified, assumes a single-threaded implementation.

-n=NUM

request a random image segment of picture NUM from the web server. Valid values are 1, 2 and 3. Default value is set to 1.

OUTPUT FORMAT

The concated image is output to a PNG file with the name of all.png.

EXAMPLES

```
paster -t 6 -n 2
```

Use 6 threads to simultaneously download all image segments of picture 2 from multiple web servers and concatenate these segments to restore picture 2. Output the concatenated picture to disk and name it all.png.

EXIT STATUS

paster exits with status 0 upon success and nonzero upon error.

2.4 Programming Tips

2.4.1 The libcurl API

Though the image segment download code using `libcurl` is provided, familiarize yourself with the `libcurl` API will help you understand the provided code. The `libcurl` documentation URL is <https://curl.haxx.se/libcurl>. The man page of each function in the `libcurl` API can be found at URL <https://curl.haxx.se/libcurl/c/allfuncs.html>.

Note the provided example `cURL` code downloads the received image segment to memory and then output the data in memory to a PNG file. The output to a PNG file is just to make it easier for you to view the downloaded image segment to help you understand the example code. However your `paster` program does not need to output each segment received to a file. An efficient way (i.e. without unnecessary file I/O) is to directly use the received image segment data in memory instead of outputting the data to a file first and then reading the data back from file into memory.

Thread Safety

`Libcurl` is thread safe but there are a few exceptions. The man page of `libcurl-thread(3)` (see <https://curl.haxx.se/libcurl/c/threadsafe.html>) is the ultimate reference. We re-iterate key points from `libcurl` manual that are relevant to this lab as follows:

- The same `libcurl` handle should not be shared in multiple threads.
- The `libcurl` is thread safe but does not have internal thread synchronization mechanism. You will need to take care of the thread synchronization.

2.4.2 The pthreads API

The `pthread(7)` man page gives an overview of POSIX threads and should be read. The SEE ALSO section near the bottom of the man page lists functions in the API. The man pages of `pthread_create(3)`, `pthread_join(3)` and `pthread_exit(3)` provide detailed information of how to create, join and terminate a thread.

The pthread Memory Leak Bug

There is a known memory leak bug related to `pthread_exit()`. Please refer to https://bugzilla.redhat.com/show_bug.cgi?id=483821 for details. Using `return()` instead of `pthread_exit()` will avoid the memory leak bug.